

AI SaaS Platform – Project Guide Report

1. Introduction

This project is a **full-stack AI SaaS (Software as a Service) platform** that provides multiple AI-powered tools under a single dashboard. The platform allows users to sign up, receive free credits, use AI tools (text and image based), preview results in real time, and upgrade their plan using an online payment gateway.

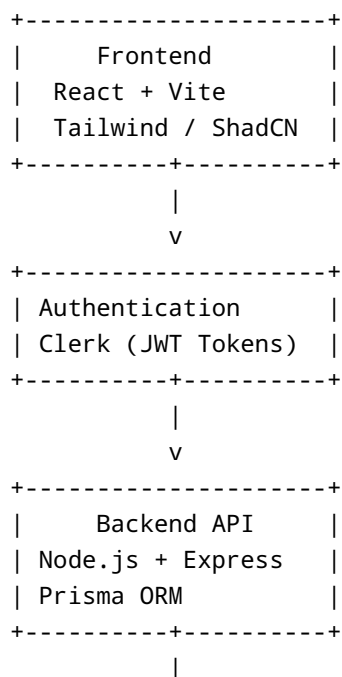
The goal of this project is to design a **modern, scalable, and future-proof web application** using industry-standard technologies while keeping the implementation easy to maintain and extend.

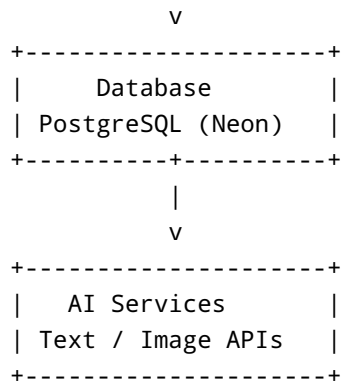
2. Project Objectives

- Build a modern AI-based SaaS platform
 - Provide multiple AI tools from a single dashboard
 - Implement secure authentication
 - Introduce a credit-based usage system
 - Enable subscription and credit top-up via payment gateway
 - Design a clean and professional UI/UX
 - Make the system scalable for future AI tools
-

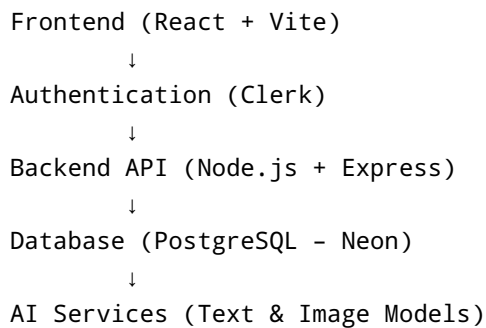
3. Overall System Architecture

3.1 System Architecture Diagram





This layered architecture ensures separation of concerns, scalability, and security.



Each layer is independent and communicates through secure APIs, making the system modular and scalable.

4. Frontend Technologies

4.1 Core Frontend Stack

Technology	Purpose
React	UI development
Vite	Fast build tool
Tailwind CSS	Styling and responsive design
ShadCN/UI	Pre-built UI components
React Router	Client-side routing
Zustand	Global state management
React Query	API data fetching and caching
Lucide React	Icons

4.2 Authentication (Frontend)

- **Clerk React SDK** is used for:
 - Sign up / Sign in
 - Session management
 - Protected routes
 - User profile handling

No password handling is required on the application side, improving security.

4.3 Frontend Folder Structure

```
src/  
├── pages/  
│   ├── Home.jsx  
│   ├── Pricing.jsx  
│   ├── Dashboard.jsx  
│   └── tools/  
│       ├── ImageGenerator.jsx  
│       ├── ArticleWriter.jsx  
│       ├── BlogWriter.jsx  
│       └── BgRemover.jsx  
├── components/  
│   ├── Sidebar.jsx  
│   ├── Navbar.jsx  
│   ├── ToolLayout.jsx  
│   ├── ToolInput.jsx  
│   └── ToolPreview.jsx  
├── store/  
│   └── creditStore.js  
├── api/  
│   └── aiApi.js  
└── main.jsx
```

5. UI / UX Design Strategy

5.1 Homepage Structure

- Hero Section (intro + CTA)
- AI Tools showcase (card grid)
- Credit system explanation
- Testimonials
- Pricing & subscription plans

- Contact section
- Image / gallery section

5.2 Dashboard Design

- Left Sidebar: AI tools navigation
- Main Content Area:
 - Left panel: Input form
 - Right panel: Live preview / output

This layout is consistent across all AI tools for better user experience.

6. Backend Technologies

6.1 Core Backend Stack

Technology	Purpose
Node.js	JavaScript runtime
Express.js	Backend framework
Prisma	ORM (Database interaction)
PostgreSQL	Relational database
Neon	Serverless PostgreSQL hosting
Stripe	Payments and subscriptions
Clerk Backend SDK	Authentication verification

7. Database Design

7.1 Database Choice

- **PostgreSQL (Neon)** is used as the database
- **Prisma** is used as the ORM

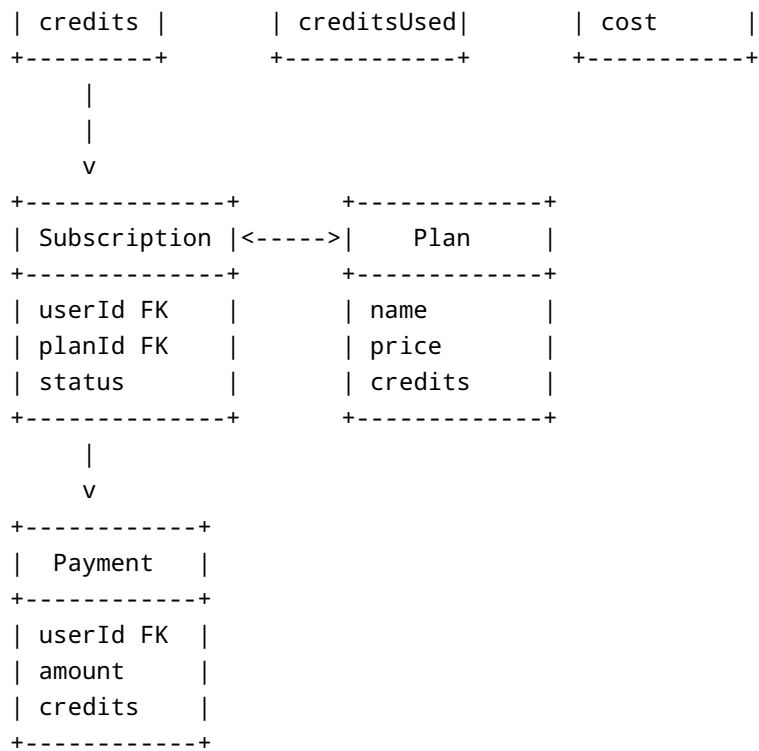
This combination provides: - Easy querying - Safe updates - Strong consistency - Easy scalability

7.2 ER Diagram (Entity Relationship Diagram)

```

+-----+      +-----+      +-----+
|  User  |      |  Usage  |      | AiTool  |
+-----+      +-----+      +-----+
| id PK  |<----->| userId FK |      | id PK  |
| email  |      | toolId FK |<----->| slug   |

```



7.1 Database Choice

- **PostgreSQL (Neon)** is used as the database
- **Prisma** is used as the ORM

This combination provides: - Easy querying - Safe updates - Strong consistency - Easy scalability

7.2 Database Tables

User

- id
- clerkUserId
- email
- credits
- createdAt

AiTool

- id
- name
- slug
- description
- creditCost
- isActive

Usage

- id
- userId
- toolId
- creditsUsed
- createdAt

Plan

- id
- name
- monthlyCredits
- price

Subscription

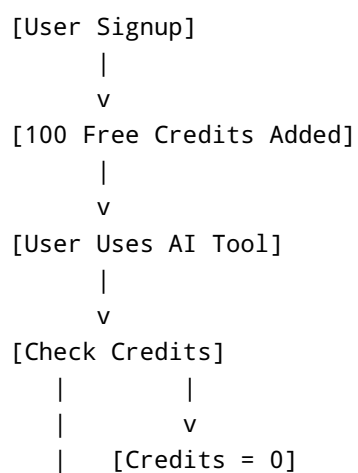
- id
- userId
- planId
- stripeSubscriptionId
- status
- expiresAt

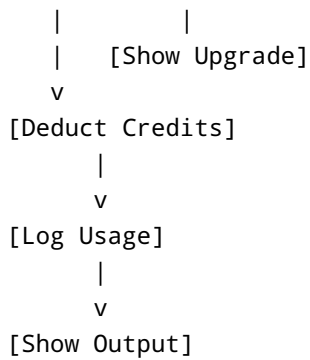
Payment

- id
- userId
- stripePaymentId
- creditsAdded
- amount
- createdAt

8. Credit-Based System

8.1 Credit Usage Flowchart





Credit Flow Explanation

1. New user receives 100 free credits
2. Each AI tool consumes predefined credits
3. Credits are checked before execution
4. Usage is logged in database
5. If credits reach zero, tools are locked

Credit Flow

1. New user signup → 100 free credits
2. AI tool usage → credits deducted
3. Credits = 0 → tools locked
4. Subscription / payment → credits added

Credits are deducted using database transactions to avoid data inconsistency.

9. AI Tools Architecture

Unified API Endpoint

```
POST /api/ai/:toolSlug
```

- Tool is identified by `toolSlug`
- Credit cost is fetched from database
- Credits are deducted
- AI service is executed
- Output is returned to frontend

This design allows easy addition of new AI tools.

10. Payment Gateway Integration

Stripe Features Used

- Secure checkout
- Subscriptions
- One-time credit purchases
- Webhooks for payment confirmation

Payment Flow

1. User selects plan
 2. Redirected to Stripe Checkout
 3. Payment success
 4. Credits added to account
 5. User redirected to dashboard
-

11. Security Considerations

- Authentication handled by Clerk
 - JWT verification on backend
 - No password storage
 - Secure Stripe webhooks
 - Database transactions for credits
-

12. Future Scalability

- New AI tools can be added without schema changes
 - Backend supports modular services
 - Frontend uses reusable layouts
 - Database design supports analytics and reporting
-

13. Conclusion

This project is designed as a **real-world AI SaaS platform** with modern UI, secure authentication, scalable backend architecture, and a flexible credit-based system. The selected technologies ensure ease of development, high performance, and future extensibility, making this project suitable for both academic and professional use.